

Credit Card & User Data Analysis

Milestone 1, 2 & 3 (Cleaning, EDA and Customer Segmentation)

FSDA Intermediate Python for Data Analysis

Reinaldy Zulfananda Arkaan

RevoU Full Stack Data Analytics | [W7W8W9 JUN26] | Intermediate

LinkedIn: <https://www.linkedin.com/in/reinaldy-zulfananda-arkaan-30567a179>

MILESTONE 1

Data Cleaning and Preparation

FSDA Intermediate Python for Data Analysis

Business Understanding & Goal

Dataset Overview

df_card

5,599 rows × 14 columns, card level records covering brand, credit limit, expiry, transaction and fraud counts.

df_user

2,000 rows × 8 columns, user profile records covering birthdate, income, debt, and credit score.

Relationship

df_card.client_id links to df_user.id, with many cards per user.

Goal of the Analysis

Summarise the last 6 months of card sales performance and build user personas from the existing customer base, by combining df_user (who the customer is) with df_card (how their cards are used), so the PM team can see which customer groups still have room to transact more. Milestone 1 delivers the foundation for that: a clean, correctly typed and deduplicated dataset covering active cards only, plus three persona attributes - age for life stage, retired_flag for earning power, and DTI for spending headroom.

This serves the PM team's business problem: finding which customer groups still have room to transact more, so campaigns can be aimed at them rather than at everyone.

Data Dictionary Overview

df_card (5,599 × 14)

Column	Description
id / client_id	Card ID / customer ID (links to df_user)
card_brand	Card brand; contains typos ("Visa ", "Jcb")
expires	Expiry month/year
credit_limit	Credit limit (text, "Rp" prefix)
count/ amt*_trx_L6M	Transaction & fraud count/amount, last 6 months

df_user (2,000 × 8)

Column	Description
id	Customer ID, links to df_card.client_id
birthdate	Date of birth
retirement_age	Customer's stated retirement age
yearly_income / total_debt	Text, "Rp" prefix
credit_score	Numeric credit score

1 4a. Card Data (Convert Data Types)

Syntax

```
# 1. Identifiers -> string
ids = ['id', 'client_id', 'card_number', 'cvv']
df_card[ids] = df_card[ids].astype(str)

# 2. Dates -> datetime
expires          -> to_datetime + MonthEnd(0)
acct_open_date   -> to_datetime

# 3. Money -> float (strip 'Rp' and '.')
credit_limit, amt_nonfraud_trx_L6M,
amt_fraud_trx_L6M -> astype(float)

# 4. Count data -> integer
count_nonfraud_trx_L6M,
count_fraud_trx_L6M -> astype('Int64')
```

Output

Column	Before	After
id, client_id, card_number, cvv	int64	string
expires, acct_open_date	"04/2030"	datetime64
credit_limit, amt_*_trx_L6M	"Rp53.189.000"	53189000.0

Insight

All four type rules from the brief are applied. Identifiers (id, client_id, card_number, cvv) become string — no arithmetic is ever done on them. Both date columns become datetime; expires is pushed to month end, because a card stays valid through the last day of its expiry month. All three money columns are stripped of 'Rp' and the '.' separator and cast to float. The two count columns become integer, using nullable Int64 so their blanks survive until 4c.

2 4b. Card Data (Handle Typos)

Syntax

```
print(
    df_card['card_brand']
    .value_counts()
)

# Fix whitespace AND casing
# (' Visa ' -> 'Visa', 'Jcb' -> 'JCB')
df_card['card_brand'] = (
    df_card['card_brand'].str.strip()
    .replace({'Jcb': 'JCB'})
)
```

Output

Value (before)	Count
Mastercard	2,826
Visa	2,093
Amex	402
JCB	206
"Visa " (trailing space)	69
"Jcb" (case typo)	3

Insight

`value_counts()` showed 6 raw values instead of 4: a trailing space ('Visa ' vs 'Visa') and a casing typo ('Jcb' vs 'JCB'). `str.strip()` alone fixes only the space and still leaves 5 categories, so the casing is mapped explicitly to reach 4 clean brands. Without this step any later grouping by `card_brand` would split the same brand across rows and understate its share.

3 4c. Card Data (Duplicates & Missing Values)

Syntax

```
print("Rows:", len(df_card), "| Unique id:", df_card['id'].nunique())
print("Duplicated data count:",
df_card.duplicated(subset=['id']).sum())

df_card = df_card.drop_duplicates(subset=['id'], keep='first')
print("Shape after drop duplicates:", df_card.shape)

print("\nMissing values:\n", df_card.isnull().sum())

df_card = df_card.fillna(0)

count_cols = ['count_nonfraud_trx_L6M', 'count_fraud_trx_L6M']
df_card[count_cols] = df_card[count_cols].astype('int64')
```

Output

Check	Result
Total rows	5,599
Unique card id	5,568
Duplicated id (removed)	31
Missing - fraud columns	5,021
Missing after fillna(0)	0

Insight

The duplicate check is based on id, the unique card identifier: one card must appear exactly once, so 5,599 rows against only 5,568 unique ids means 31 records were entered twice. Checking id matters a plain duplicated() across all columns returns 0 here, because one copy of each pair carries a trailing space in card_brand. Duplicates are deleted, not replaced: a twin row adds no new information and would double-count transactions. Missing values are filled with 0, since a blank means no transaction occurred.

4 4d. Card Data (Remove Expired & 0-Limit Cards)

Syntax

```
cutoff = pd.to_datetime('2025-05-31')
condition1 = (
    df_card['expires'] > cutoff)
condition2 = (
    df_card['credit_limit'] > 0)

df_card = df_card[
    condition1 & condition2
]

print(df_card.shape)
```

Output

Metric	Value
Rows before	5,568
Rows after	5,528
Rows removed	40 (~0.7%)

Insight

40 cards are excluded because they were already expired as of the May 31, 2025 cutoff, or had a credit_limit of 0, out of the 5,568 rows remaining after the 31 duplicated id were removed in step 4c. These cards are inactive and not relevant for analyzing active card transaction behavior.

5 5a. User Data (Convert Data Types)

Syntax

```
df_user['id'] = (
    df_user['id'].astype(str))

df_user['birthdate'] = pd.to_datetime(
    df_user['birthdate'],
    format='%Y-%m-%d'
)

for col in ['per_capita_income',
            'yearly_income',
            'total_debt']:
    df_user[col] = (
        df_user[col].astype(str)
        .str.replace('Rp', '')
        .str.replace('.', '')
        .astype(float))
```

Output

Column	Before	After
id	int64	string
birthdate	"1972-11-25"	datetime64
per_capita_income, yearly_income	"Rp45.937.000"	45937000.0
total_debt	"Rp38.138.095"	38138095.0

Insight

id becomes string as a customer identifier, and birthdate becomes datetime so age can be computed against the cutoff. All three money columns are converted, not only the two used for DTI: per_capita_income was still text, and it is needed later to compare a customer's income against the income level of their area — a useful persona signal. retirement_age and credit_score are already integers and are left unchanged.

6 5b. User Data (Add Age, Retired Flag, DTI)

Syntax

```
cutoff = pd.to_datetime('2025-05-31')
df_user['age'] = round(
    (cutoff - df_user['birthdate'])
    .dt.days / 365.25
).astype(int)

df_user['retired_flag'] = np.where(
    df_user['age'] >=
    df_user['retirement_age'], 1, 0)

df_user['DTI'] = (
    df_user['total_debt']
    / df_user['yearly_income']
)
```

Output

age	ret_age	flag	DTI
53	66	0	0.41
52	68	0	0.47
81	67	1	0.00
62	63	0	0.15
43	70	0	0.32

Insight

age is computed against the May 31, 2025 cutoff so every figure stays reproducible. retired_flag compares each user's age with their OWN retirement_age, which ranges from 50 to 79 — a fixed threshold of 55 would flag 599 users as retired, while the per-user rule flags 290. Row 4 shows why it matters: age 62 against a retirement age of 63 is not retired yet. DTI (total_debt / yearly_income) shows how much headroom a customer still has to spend.

Data Cleaning Summary

Data Cleaning Step	Original Data	After Cleaning	Reason
Convert data type (4a, 5a)	IDs numeric; dates, money and counts as text	IDs string; dates datetime; money float; counts int	Match each column to what it represents, so dates can be compared and money can be summed
Handle typos (4b)	card_brand: 6 raw values ('Visa ', 'Jcb')	card_brand: 4 clean categories	Whitespace and casing split the same brand into extra categories
Delete duplicates (4c)	5,599 rows but only 5,568 unique card id -> 31 duplicated id	5,568 rows (31 removed)	id is the unique card identifier, so a repeated id means a re-entered card. Deleted, not replaced: a twin row would double-count transactions
Treat missing values (4c)	Up to 5,021 missing (fraud columns)	0 missing (filled with 0)	Blank = no transaction/fraud occurred in that period
Remove expired/0-limit cards (4d)	5,568 rows	5,528 rows	Inactive cards irrelevant to active transaction behavior
Add age, retired_flag, DTI (5b)	8 columns	11 columns	Support persona building (life stage, earning power, spending headroom)

Summary of Key Findings

- **Final active card dataset:** 5,528 cards remain after removing 71 rows overall from the original 5,599 (31 duplicated card id plus 40 expired or 0 credit limit cards), about 1.3% of the original dataset.
- **Card brand:** The inconsistent values such as trailing spaces and different capitalization were cleaned, reducing 6 raw categories to 4 consistent brands: Mastercard, Visa, Amex, and JCB.
- **Duplicates and missing values:** 31 duplicated card id were found and removed from df_card. The check is based on id, the unique card identifier: 5,599 rows carried only 5,568 unique ids. Most missing values were in the fraud related columns, with up to 90.2% missing.
- **User data:** df_user was updated with three new features: age, retired_flag, and DTI, as the attributes the personas will be built on. retired_flag is compared against each user's own retirement_age (which ranges 50-79), giving 290 retired customers.
- **Final dataset:** Both df_card and df_user are now cleaned and ready for the next stage: summarising sales performance and building the user personas.

MILESTONE 2

Exploratory Data Analysis

FSDA Intermediate Python for Data Analysis

a

Total Net Profit (Last 6 Months)

Syntax

```
# MDR margin: Rp1,5 mio profit per
# Rp100 mio of transactions -> 1.5%
MDR_RATE = 0.015

# fraud is NOT a real sale, so it is
# excluded from the revenue base
total_sales = df_card[
    'amt_nonfraud_trx_L6M'].sum()
total_fraud = df_card[
    'amt_fraud_trx_L6M'].sum()

mdr_profit = MDR_RATE * total_sales
net_profit = mdr_profit - total_fraud

print(f"NET PROFIT (6 months): "
      f"Rp {net_profit:,.0f}")
```

Output

Component	Value
MDR fee rate	1.50%
Total legitimate sales	Rp 455.863.486.600
MDR fee profit	Rp 6.837.952.299
Total fraud amount	- Rp 1.013.378.300
NET PROFIT (6 months)	Rp 5.824.573.999

Insight

Net profit for the half year is Rp 5,82 miliar, so the portfolio is profitable. Fraud is not a rounding error: Rp 1,01 miliar of losses wipes out 14,8% of the Rp 6,84 miliar gross MDR revenue. Because the MDR margin is only 1,5%, every Rp 1 lost to fraud needs about Rp 67 of extra legitimate spending to replace it, which makes fraud control the fastest lever on profit.

b Fraud Rate of RevoBank (Syntax & Output)

Syntax

```
# Step 1: totals from the cleaned card data
total_fraud    = df_card['amt_fraud_trx_L6M'].sum()
total_nonfraud = df_card['amt_nonfraud_trx_L6M'].sum()
total_all      = total_fraud + total_nonfraud

# Step 2: fraud rate = share of total value
fraud_rate = total_fraud / total_all
print(f"Fraud Rate: {fraud_rate:.2%}")

# Step 3: pie chart of the proportion
plt.figure(figsize=(6, 6))
plt.pie(
    [total_fraud, total_nonfraud],
    labels=['Fraud', 'Non-Fraud'],
    autopct='%1.2f%%',
    colors=['#e74c3c', '#2ecc71'],
    startangle=90,
    explode=(0.08, 0)
)
plt.title('Fraud vs Non-Fraud Amount Proportion')
plt.show()
```

Output

Metric	Value
Total fraud amount	Rp 1.013.378.300
Total non-fraud amount	Rp 455.863.486.600
Total transaction value	Rp 456.876.864.900
Fraud rate (6 months)	0,22%
Safe benchmark	< 1%

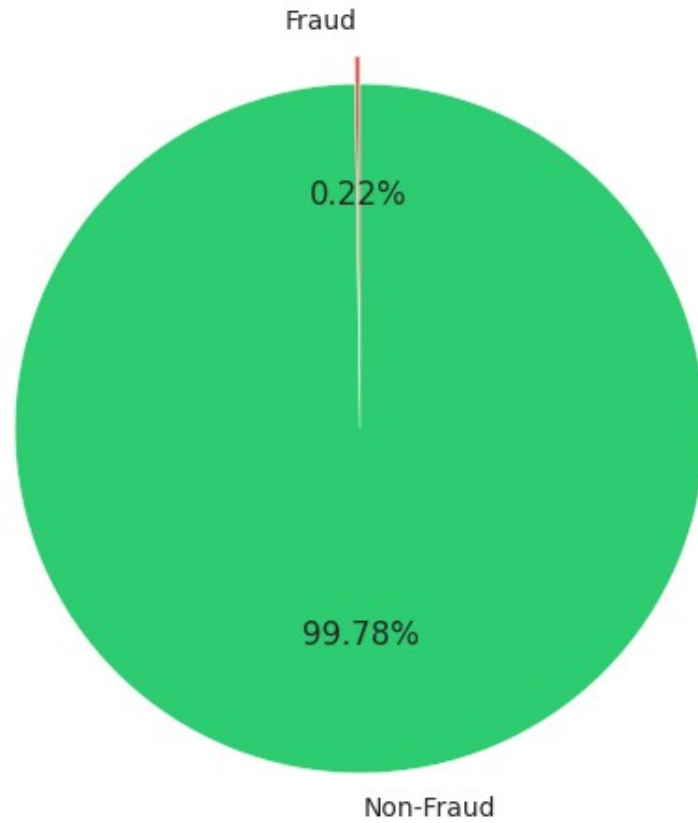
Method

Fraud rate is the total fraud amount divided by the total of fraud plus non-fraud amount, so it measures the share of transaction value lost to fraud, not the share of transactions. The fraud slice is exploded in the pie chart so a share this small stays visible.

b Fraud Rate of RevoBank (Visualisation & Insight)

Visualisation

Fraud vs Non-Fraud Amount Proportion (Last 6 Months)



What the chart shows

Non-fraud is 99,78% of transaction value, fraud only 0,22%.

In rupiah: Rp 1,01 miliar of fraud against Rp 455,86 miliar of legitimate sales.

The fraud slice sits well below the 1% level usually treated as safe.

Insight

At 0,22% of transaction value the fraud rate is well inside the safe range, so RevoBank's exposure is still under control. The caveat is scale: even 0,22% already costs Rp 1,01 miliar in six months, and absolute losses grow with volume. Promos can be scaled, but velocity checks and stronger authentication on high-value transactions should stay in place so the rate does not drift up.



Transaction Behaviour per Card Brand (Syntax & Output)

Syntax

```
# card-level data, before the merge,
# so the card_brand label is preserved
brand_behaviour = df_card.groupby('card_brand').agg(
    avg_trx_count=('count_nonfraud_trx_L6M', 'mean'),
    avg_trx_amount=('amt_nonfraud_trx_L6M', 'mean')
).round(2).sort_values(
    'avg_trx_amount', ascending=False)

print(brand_behaviour)

# two bar charts side by side
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

brand_behaviour['avg_trx_count'].plot(
    kind='bar', ax=ax[0], color='#3498db')
ax[0].set_title('Average Transaction Count')

brand_behaviour['avg_trx_amount'].plot(
    kind='bar', ax=ax[1], color='#9b59b6')
ax[1].set_title('Average Transaction Amount')

plt.tight_layout()
plt.show()
```

Output

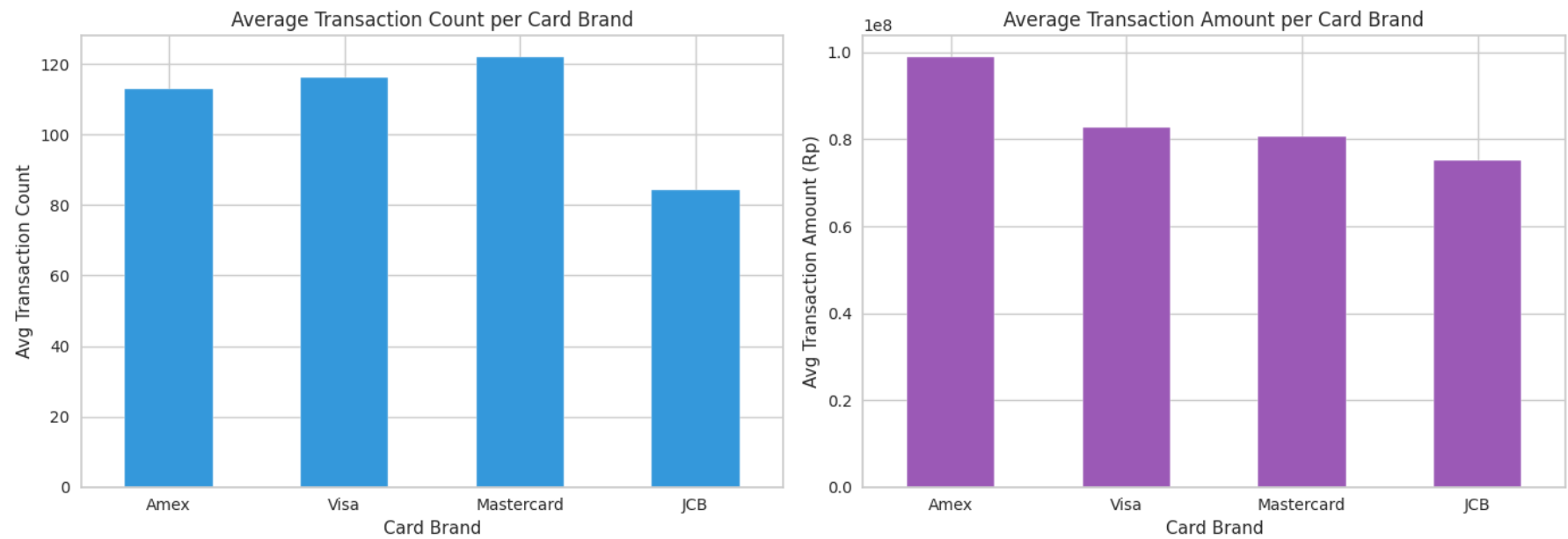
Card Brand	Avg Trx Count	Avg Trx Amount (Rp)
Amex	112,94	98.852.996
Visa	116,06	82.617.652
Mastercard	121,93	80.595.841
JCB	84,40	75.026.117

Method

The average is used rather than the sum, so a brand is not ranked higher simply because it has more cards in the portfolio. The calculation runs on card-level data before the merge, because card_brand only exists there. Sorting by average amount puts the highest-value brand first.



Transaction Behaviour per Card Brand (Visualisation & Insight)



Insight & Recommendation

The two leaders are different brands. Amex has the highest value per card (Rp 98,9 juta on average) but the second lowest frequency, while Mastercard is the most active (121,9 transactions) at a lower ticket size. JCB trails on both and is the weakest brand. Recommendation: premium offers and higher limits for Amex, frequency-driven promos such as per-transaction cashback for Mastercard and Visa, and an activation campaign for JCB.

d DTI: Retired vs Non-Retired (Syntax & Output)

Syntax

```
# d. DTI comparison: retired (1) vs non-retired (0)
dti_by_retired = df_user.groupby('retired_flag')['DTI'].describe().round(4)
print(dti_by_retired)

# Visualize with a boxplot
plt.figure(figsize=(8, 5))
sns.boxplot(data=df_user, x='retired_flag', y='DTI', hue='retired_flag',
            palette='Set2', legend=False)
plt.title('DTI Distribution: Retired vs Non-Retired Users')
plt.xlabel('Retired Flag (0 = Non-Retired, 1 = Retired)')
plt.ylabel('Debt-to-Income Ratio (DTI)')
plt.show()

# --- Auto-insight (real numbers) ---
med = df_user.groupby('retired_flag')['DTI'].median().to_dict()
med_non, med_ret = med.get(0, float('nan')), med.get(1, float('nan'))
if med_ret > med_non:
    concl = "retired users carry a HIGHER median DTI, so they are
marginally riskier on this metric"
else:
    concl = "retired users are NOT riskier; their median DTI is similar to
or lower than non-retired users"
print("\n--- Insight ---")
print(f"Median DTI -> non-retired: {med_non:.4f} | retired: {med_ret:.4f}")
print(f"Conclusion: {concl}.")
```

Output

Statistic	Non-Retired (0)	Retired (1)
Median DTI	0,30	0,14
Q1 (25%)	0,22	0,04
Q3 (75%)	0,39	0,30
Whisker range	0,00 to 0,65	0,00 to 0,70

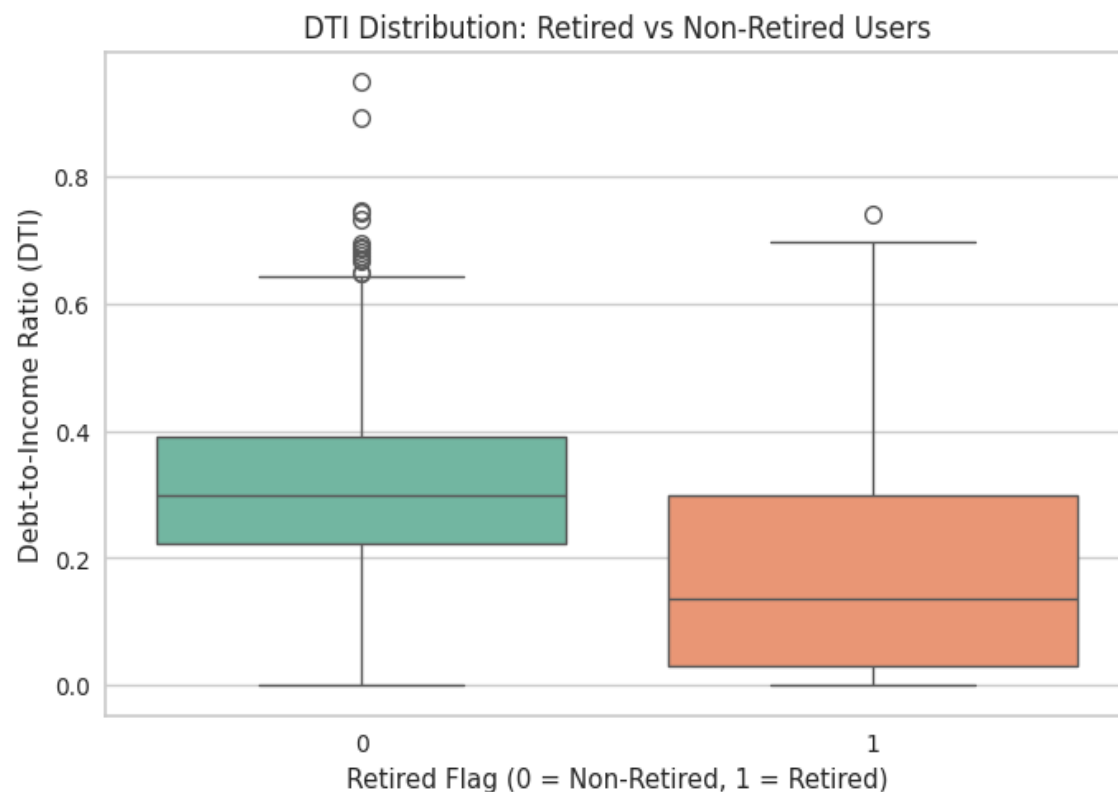
Method

retired_flag was built in Milestone 1 with a single cut-off: age 55 and above is flagged as retired. DTI is total_debt divided by yearly_income. The table reports the quartiles from describe() instead of the mean, because DTI is right skewed by outliers reaching 0,95, which drags the mean away from the typical user.

d

DTI: Retired vs Non-Retired (Visualisation & Insight)

Visualisation



What the chart shows

Median DTI is 0,30 for non-retired users and 0,14 for retired users, so retired users carry roughly half the debt burden.

The retired box is lower but wider (Q1 0,04 to Q3 0,30), so the group is more spread out, not more indebted.

Non-retired users show many high outliers, up to DTI 0,95, which is why the median is used instead of the mean.

Insight

Retired users are NOT riskier on this metric. Their median DTI is 0,14 against 0,30 for non-retired users, and their whole box sits below the non-retired median. Lower yearly income pushes DTI up, but far less remaining debt pulls it down harder, and the net effect is a lighter debt burden. Risk is concentrated in the non-retired group instead, where the upper tail runs past DTI 0,90. Recommendation: do not tighten credit on retirement status alone, and screen on DTI and credit score instead.

Merge Card & User Data

Syntax

```
# one row per client_id
df_card_agg = df_card.groupby('client_id').agg(
    credit_limit=('credit_limit', 'sum'),
    days_since_last_trx=(
        'days_since_last_trx', 'min'),    # recency
    count_nonfraud_trx_L6M=(
        'count_nonfraud_trx_L6M', 'sum'),
    amt_nonfraud_trx_L6M=(
        'amt_nonfraud_trx_L6M', 'sum'),
    count_fraud_trx_L6M=(
        'count_fraud_trx_L6M', 'sum'),
    amt_fraud_trx_L6M=(
        'amt_fraud_trx_L6M', 'sum')
).reset_index()

# then merge onto the user table
df_user_agg = df_user.merge(
    df_card_agg, left_on='id',
    right_on='client_id', how='inner'
).drop(columns=['client_id'])
```

Output

Step	Result
df_card (cleaned)	5.528 rows
df_card_agg (per client_id)	1.938 × 7
df_user (cleaned)	2.000 rows
df_user_agg (merged)	1.938 × 17
Recency rule	min(days_since_last_trx)
Money & count rule	sum across all cards

Insight

Aggregating before merging is what keeps the result at user level. Merging the raw card rows first would have inflated 2.000 users into roughly 5.500 duplicated rows and double counted every user holding more than one card. Recency takes the minimum because the most recent transaction is the smallest number of days. The inner join returns 1.938 of 2.000 users: the other 62 hold no active card after the Milestone 1 cleaning.

Average DTI per Credit Score Category (Syntax & Output)

Syntax

```
# Step 1: bucket the credit score
bins = [0, 580, 670, 740, 800, 851]
labels = ['Poor', 'Fair', 'Good',
          'Very Good', 'Exceptional']

df_user_agg['credit_score_category'] = pd.cut(
    df_user_agg['credit_score'],
    bins=bins, labels=labels, right=False)

# Step 2: average DTI + transaction behaviour
category_summary = df_user_agg.groupby(
    'credit_score_category', observed=True).agg(
    avg_dti=('DTI', 'mean'),
    total_trx_count=('count_nonfraud_trx_L6M', 'sum'),
    total_trx_amount=('amt_nonfraud_trx_L6M', 'sum')
).round(4)

print(category_summary)

# Step 3: bar chart per category
sns.barplot(x=category_summary.index,
            y=category_summary['avg_dti'],
            palette='viridis')
```

Output

Category	Avg DTI	Trx Count	Trx Amount
Poor	0,3018	23.522	Rp 17,6 M
Fair	0,3182	93.756	Rp 65,8 M
Good	0,2503	307.726	Rp 220,9 M
Very Good	0,2457	168.619	Rp 114,0 M
Exceptional	0,2429	56.749	Rp 37,5 M

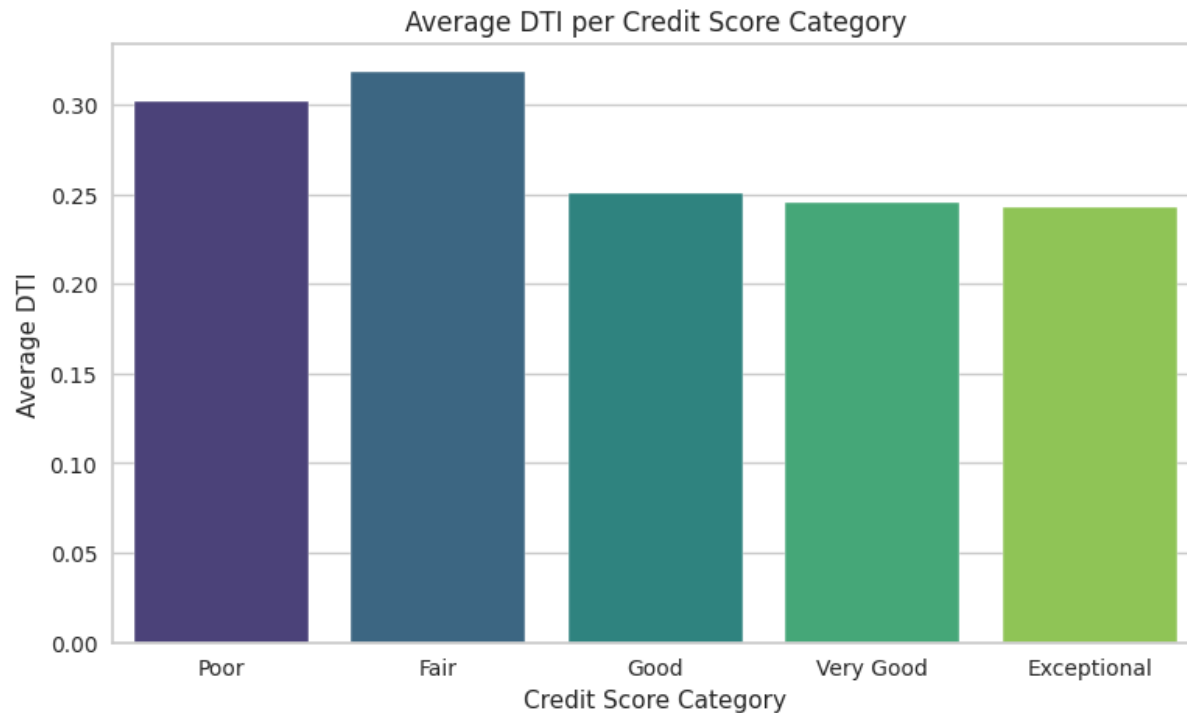
M = miliar rupiah

Method

pd.cut with right=False assigns each score to a standard bucket: Poor below 580, Fair 580 to 669, Good 670 to 739, Very Good 740 to 799, Exceptional 800 to 850. Transaction count and amount are summed per category, so debt burden and revenue contribution can be read side by side.

Average DTI per Credit Score Category (Visualisation & Recommendation)

Visualisation



What the chart shows

Average DTI is highest in Fair (0,3182) and lowest in Exceptional (0,2429).

Poor (0,3018) sits below Fair, so the ranking is not perfectly ordered.

Good carries the largest volume: Rp 220,9 miliar of transaction value.

Recommendation

Average DTI does fall as the score improves, so the score broadly reflects real debt burden, except at the bottom where Poor sits below Fair. Volume is concentrated where risk is lowest: Good to Exceptional carries about 82% of transaction value at a DTI under 0,26. Push promos and limit increases there, and serve Poor and Fair with tighter limits and closer monitoring.



User Age vs Credit Limit (Syntax & Output)

Syntax

```
# trim the volatile upper tail:
# ages above 60 hold too few users
df_age = df_user_agg[df_user_agg['age'] <= 60]

# average credit limit per age
avg_credit_by_age = df_age.groupby(
    'age')['credit_limit'].mean()

# line chart
plt.figure(figsize=(11, 5))
avg_credit_by_age.plot(
    kind='line', marker='o', color='#e67e22')
plt.title('Average Credit Limit by User Age')
plt.xlabel('User Age')
plt.ylabel('Average Credit Limit (Rp)')
plt.grid(True, alpha=0.3)
plt.show()
```

Method

Item	Detail
Source	df_user_agg (1.938 users)
Metric	mean credit_limit per age
credit_limit	summed over all cards
Scope	age ≤ 60 (tail trimmed)
Direction	positive, rising with age

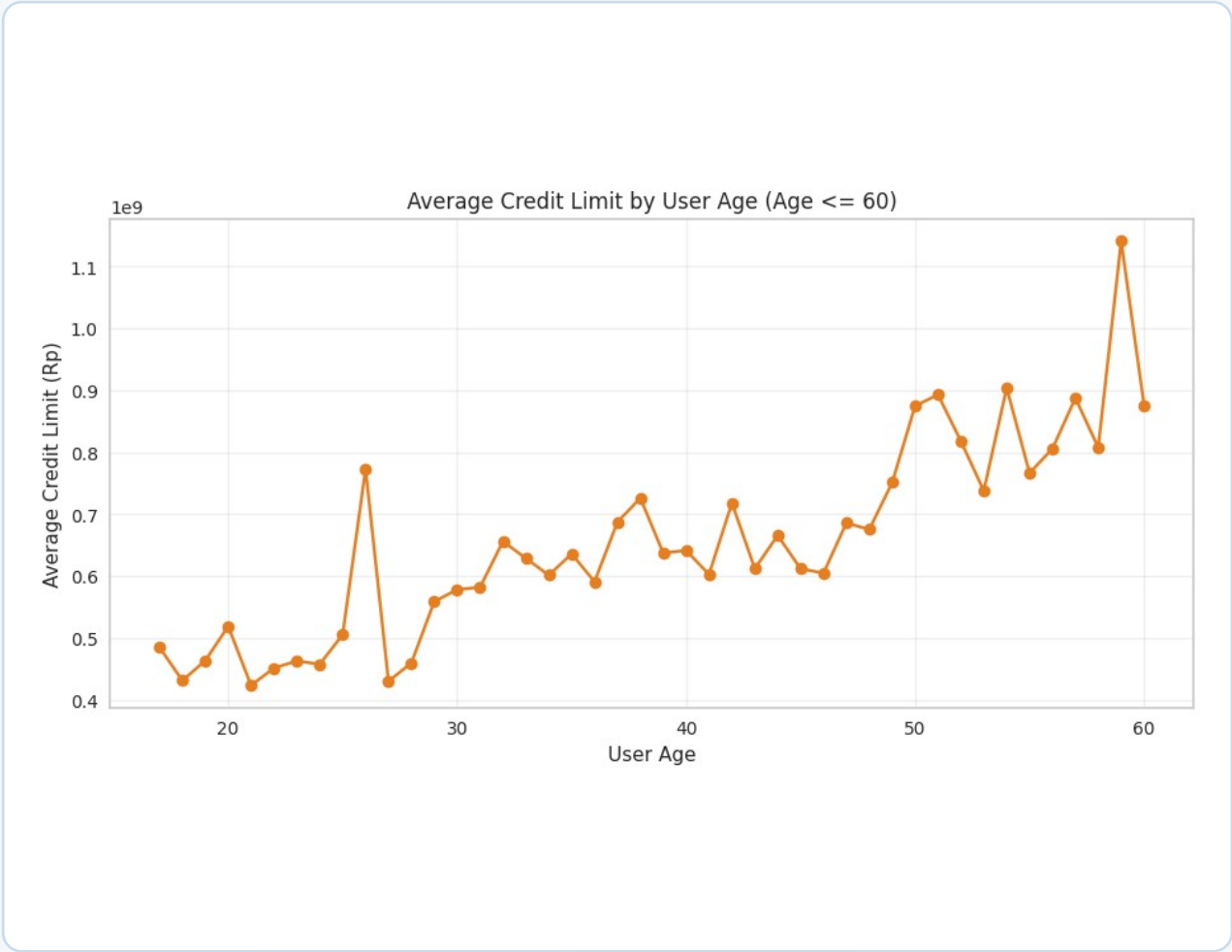
Method

credit_limit is summed per user during the merge, so it is the total limit across every card that user holds. Users above 60 are trimmed because those age groups contain few users, which makes their yearly averages jump around and hides the underlying trend.



User Age vs Credit Limit (Visualisation & Recommendation)

Visualisation



What the chart shows

Average limit rises from about Rp 0,45 miliar at age 20 to about Rp 0,85 miliar at age 60.

The line is noisy year to year, but the upward direction is consistent.

Ages above 60 are excluded because those groups are too small to be stable.

Recommendation

The relationship is positive, and it tracks income and credit history rather than spending appetite. Younger customers with a strong score are therefore capped below their real capacity. Raise limits selectively for users under 35 with a Good or better score and low DTI, but note that an unused limit earns no MDR revenue and only adds default exposure.

Summary of Key Findings

Profitable, but thin margin: net profit for the last 6 months is Rp 5,82 miliar, from Rp 6,84 miliar of MDR revenue on Rp 455,9 miliar of legitimate sales minus Rp 1,01 miliar of fraud.

Fraud is contained: the fraud rate is 0,22% of transaction value, well below the ~1% safe benchmark, yet it still erases 14,8% of gross MDR revenue.

Brands behave differently: Amex is the highest-value brand per card (Rp 98,9 juta), Mastercard the most frequently used (121,9 transactions), and JCB the weakest on both.

DTI vs retirement: retired users are not riskier, with a median DTI of 0,14 against 0,30 for non-retired users, so the long high-DTI tail sits in the non-retired group.

DTI vs credit score: average DTI falls from 0,3182 (Fair) to 0,2429 (Exceptional), and about 82% of transaction value already comes from the Good to Exceptional groups.

Age vs credit limit: the average limit roughly doubles between the early twenties and age 60, so younger high-score customers are capped below their capacity.

Rp 5,82 M

net profit in 6 months

0,22%

fraud rate, safe benchmark is 1%

82%

of value from Good to Exceptional scores

Recommendations for RevoBank

Priority	Action	Why (evidence)	Expected effect
1. Grow low-risk volume	Target promos and limit increases at the Good, Very Good and Exceptional credit score groups.	They already carry ~82% of transaction value at an average DTI below 0,26.	More MDR revenue without raising the risk profile.
2. Hold the fraud line	Keep velocity checks and step-up authentication on high-value transactions before scaling promos.	Fraud is only 0,22% of value but already removes 14,8% of gross MDR revenue.	Protects the thin 1,5% margin as volume grows.
3. Treat brands separately	Premium offers for Amex, per-transaction cashback for Mastercard and Visa, activation campaign for JCB.	Amex leads on value per card, Mastercard on frequency, JCB trails on both.	Higher ticket size and higher frequency where each brand is strongest.
4. Unlock young capacity	Selective limit review for users under 35 with a Good or better score and low DTI.	Average limit roughly doubles from the early twenties to age 60.	Removes an artificial cap on spending in an under-served group.
5. Tighten the tail	Lower limits and closer monitoring for the Poor and Fair groups, and screen on DTI rather than on retirement status.	Poor and Fair carry the highest average DTI (0,30–0,32) for only ~18% of value, while retired users sit lower at a median DTI of 0,14.	Lower expected loss with little revenue given up.

MILESTONE 3

Customer Segmentation

FSDA Intermediate Python for Data Analysis



Segmentation Method: k-means on RFM Features

Why k-means, not RFM scoring

Objective: split 1.938 card customers into a small number of behaviourally distinct segments, then identify which of them can realistically be encouraged to spend more, and which should not be pushed because their debt burden is already high.

- 1. It accepts more than three variables, so one segment can be described by spending behaviour and by capacity and risk at the same time. That is what a decision on who to push actually needs.
- 2. It finds groups from the structure of the data instead of manual score cut-offs, so a combination such as an approved limit that is never used appears on its own. Plain RFM scoring cannot see that combination.
- 3. The number of segments is validated objectively with the Elbow Method and Silhouette Analysis, while the number of RFM buckets is set by convention.

RFM is still the base of the model, so every cluster stays readable in RFM language.

The result is judged on two things: whether the clusters are statistically sound (elbow and silhouette) and whether each one leads to a different commercial action.

RFM columns used

Factor	Column	Why this column
R	days_since_last_trx	min across the user's cards, lower = more recently active
F	count_nonfraud_trx_L6M	legitimate transactions in 6 months, summed
M	amt_nonfraud_trx_L6M	legitimate value in 6 months, the exact MDR base

Supporting features

credit_limit: approved headroom, so a large limit with low spend flags a customer who can spend more without new credit.

DTI: debt burden, which keeps risk inside the model instead of outside it.

age: life stage for product and channel targeting.

Excluded from clustering and kept only for profiling: credit_score, retired_flag, gender and the fraud columns.

Feature Preparation & Scaling (Syntax & Output)

Syntax

```
feature_cols = [  
    'days_since_last_trx',    # Recency  
    'count_nonfraud_trx_L6M', # Frequency  
    'amt_nonfraud_trx_L6M',   # Monetary  
    'credit_limit', 'DTI', 'age']  
  
df_model = df_seg.set_index('id')[feature_cols]  
  
# DTI can be inf if yearly_income = 0  
df_model = df_model.replace([np.inf, -np.inf], 0)  
df_model = df_model.fillna(0)  
  
# de-skew first, then standardise  
scaler = PowerTransformer()  
scaler.fit(df_model)  
  
df_scaled = pd.DataFrame(  
    scaler.transform(df_model),  
    columns=df_model.columns,  
    index=df_model.index)  
  
print(pd.DataFrame({  
    'skew_before': df_model.skew(),  
    'skew_after':  df_scaled.skew()}).round(2))
```

Output

Feature	Skew before	Skew after
credit_limit	3,06	0,05
amt_nonfraud_trx_L6M	1,57	-0,45
count_nonfraud_trx_L6M	1,22	-0,33
days_since_last_trx	0,48	0,09
age	0,42	-0,04
DTI	0,19	-0,01

Method

The modelling table is 1.938 users × 6 features.

PowerTransformer (Yeo-Johnson) is used because credit_limit is heavily right skewed at 3,06. MinMax and Standard scaling change the range but keep the skew, which makes k-means chase outliers and produce one huge cluster plus several tiny ones.

After the transform every feature sits between -0,45 and 0,09, so no column dominates the distance calculation.

Choosing the Number of Clusters (Syntax & Output)

Syntax

```
# Elbow: inertia for k = 1..15
distortions = []
K = range(1, 16)

for n_clusters in K:
    model = KMeans(n_clusters, random_state=1000,
                   n_init='auto')
    model.fit(df_scaled)
    distortions.append(model.inertia_)

# Silhouette: k = 3..8, plus cluster balance
sil_scores = {}

for i, n_clusters in enumerate(range(3, 9)):
    model = KMeans(n_clusters, random_state=1000,
                   n_init='auto')
    model.fit(df_scaled)

    sil = SilhouetteVisualizer(
        model, is_fitted=True, ax=ax[i])
    sil.fit(df_scaled)
    sil_scores[n_clusters] = sil.silhouette_score_

    sizes = pd.Series(
        model.labels_).value_counts(normalize=True)
```

Output

k	Inertia	Drop vs previous k
1	11.628,0	-
2	5.273,5	54,6%
3	4.158,9	21,1%
4	3.626,7	12,8%
5	3.229,0	11,0%
6	3.001,2	7,1%

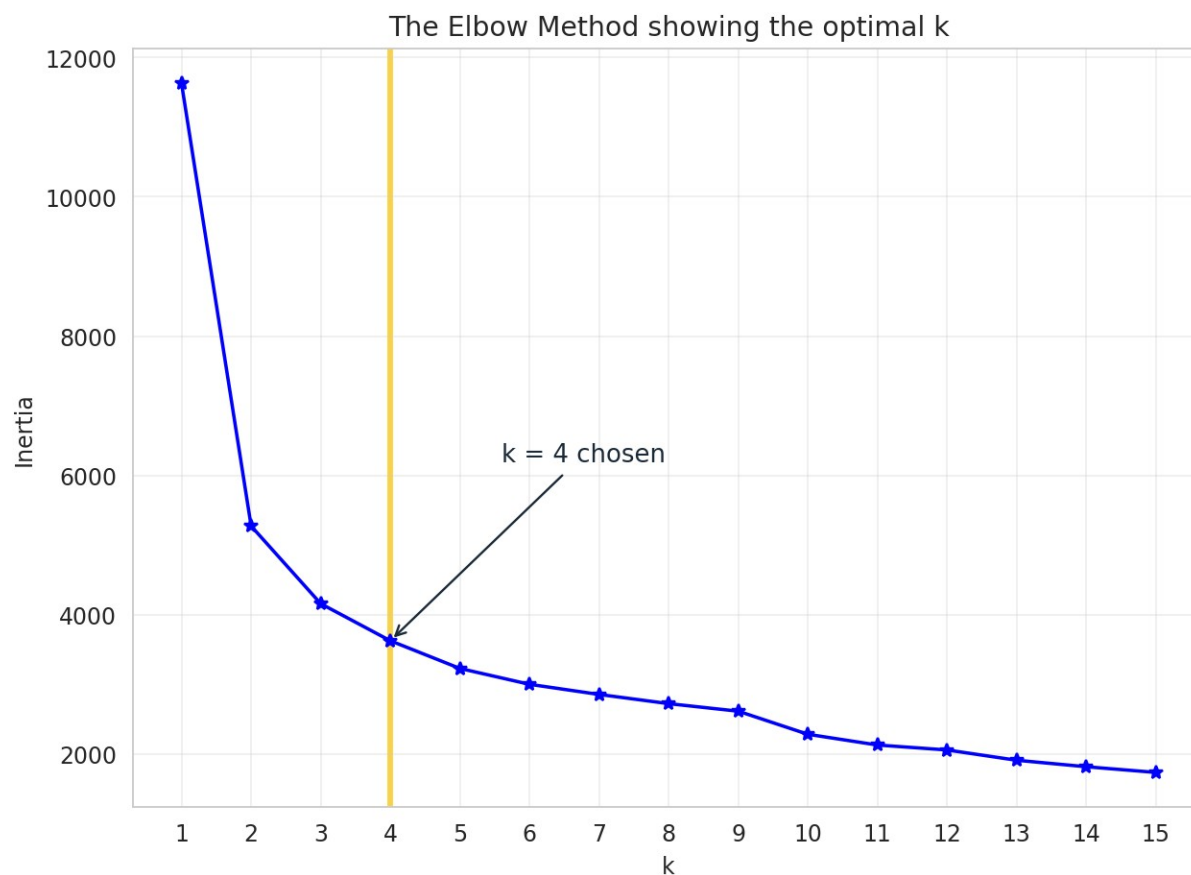
Method

Inertia always falls as k grows, so the target is the bend, not the minimum. Reading the drop column is more reliable than eyeballing the curve.

The drop stays large through k = 4 and then flattens, so the elbow sits between k = 3 and k = 4. The silhouette score decides between the two.

Elbow Method (Visualisation & Insight)

Visualisation



What the chart shows

Inertia falls from 11.628,0 at $k = 1$ to 3.626,7 at $k = 4$.

The steep part ends after $k = 3$: the drop runs 54,6%, then 21,1%, then 12,8%, and only 11,0% going to $k = 5$.

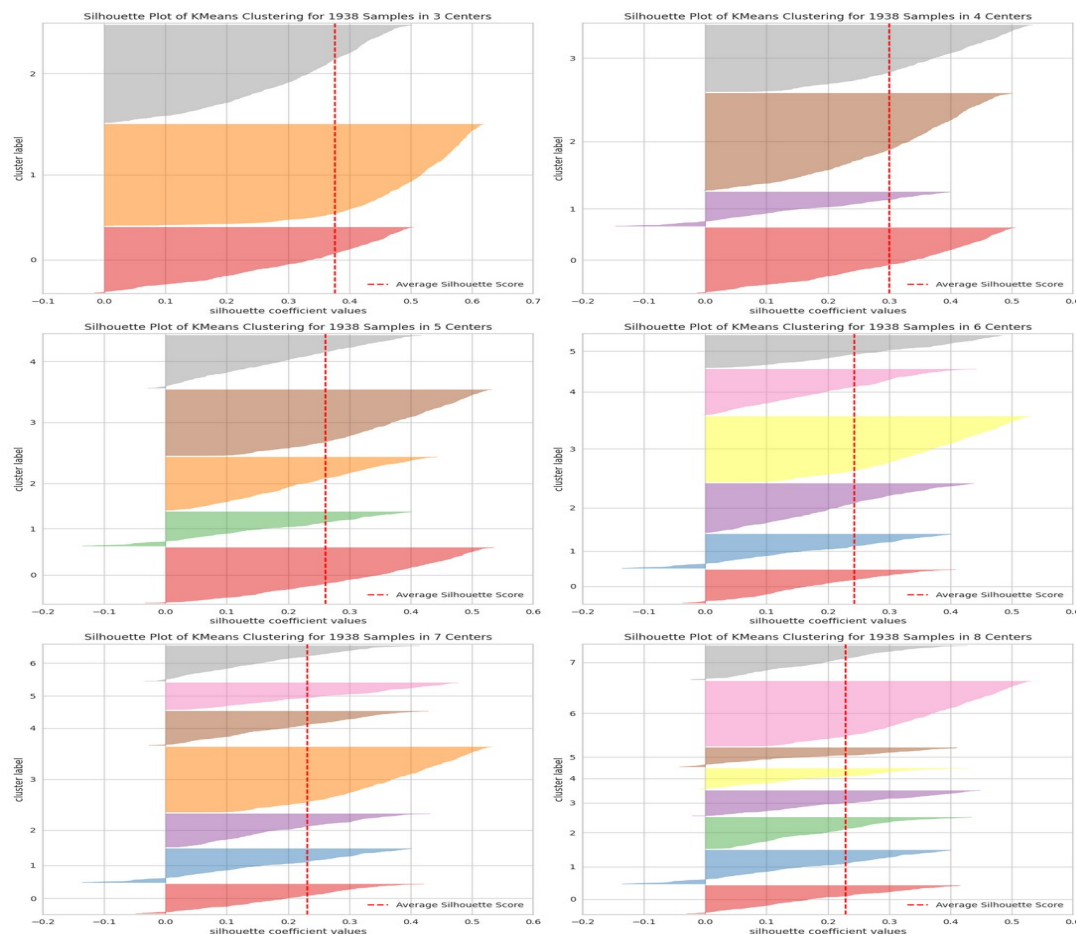
From $k = 6$ the curve is almost flat, with each extra cluster buying under 8% of inertia.

Insight

The bend sits between $k = 3$ and $k = 4$. Below that the segments are too coarse, and above it each extra cluster adds little separation while making the result harder to act on. Because the elbow does not give a single answer here, the choice between 3 and 4 is settled with the silhouette score on the next slide.

Silhouette Analysis & Final Choice of k

Visualisation



Output

k	Silhouette	Smallest	Largest
3	0,3765	24,8%	38,3%
4	0,3001	13,1%	36,9%
5	0,2617	13,2%	25,2%
6	0,2431	12,4%	25,2%
7	0,2320	10,4%	25,2%
8	0,2286	7,5%	25,2%

Decision: k = 4

k = 3 scores highest (0,3765) but merges the two inactive groups into one, which hides the difference between a dormant customer who is safe to activate and one who is already over-leveraged.

k = 4 keeps a usable score of 0,3001, every cluster holds at least 13,1% of the base, and it yields four segments that call for four different actions. From k = 5 the score keeps falling without adding a segment worth a separate campaign.



Cluster Profiling (Syntax & Output)

Syntax

```
K_OPT = 4 # from the elbow and silhouette

model = KMeans(n_clusters=K_OPT,
               random_state=1000, n_init='auto')
model.fit(df_scaled)

# labels back on the UNSCALED data,
# so the profile is in real units
df_result = df_model.copy()
df_result['cluster'] = model.labels_

cluster_profile = df_result.groupby('cluster').agg({
    'days_since_last_trx': 'median',
    'count_nonfraud_trx_L6M': 'median',
    'amt_nonfraud_trx_L6M': 'mean',
    'credit_limit': 'mean',
    'DTI': 'median',
    'age': 'median'})

print(cluster_profile.round(2))
```

Output

Cluster	Customers	Share	Credit score	Retired
0	479	24,7%	723	70,0%
1	253	13,1%	721	32,0%
2	715	36,9%	709	22,5%
3	491	25,3%	697	2,7%

Method

The median is used for the skewed behavioural columns so a handful of extreme customers does not distort the segment profile, and the mean for the money columns because total spending is what RevoBank actually earns MDR on.

No cluster is a fragment: the smallest still holds 253 of 1.938 users, so all four are large enough to run a campaign on.

Credit score and retirement status were kept out of the model and are shown here only to profile the result. Both track the segments closely, which is a good sign that the behavioural split is real.



The Four Segments (Cluster Interpretation)

Cluster & Persona	Recency (days)	Trx count	Avg spend (Rp)	Avg limit (Rp)	DTI	Age	Customers
0 - High-Value Active Spenders	6	494	419.526.599	112.548.948	0,07	64	479 (24,7%)
1 - Young Light Users	604	0	6.626	68.755.051	0,13	41	253 (13,1%)
2 - Leveraged Heavy Spenders	9	477	356.515.481	68.882.540	0,34	46	715 (36,9%)
3 - High-Burden Low Spenders	604	0	0	35.737.059	0,35	25	491 (25,3%)

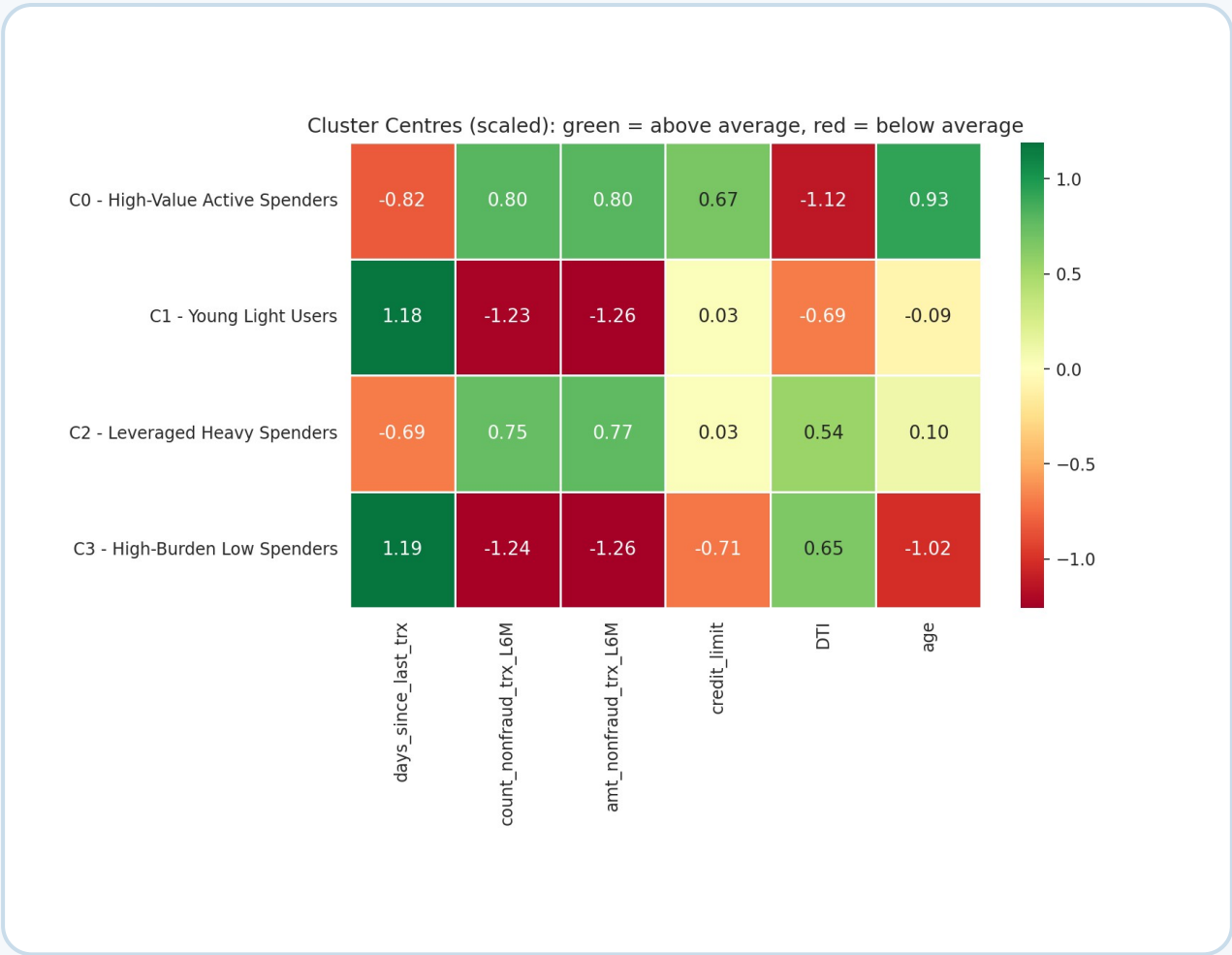
Interpretation

1. Cluster 0 - High-Value Active Spenders [R High | F High | M High | Limit High | DTI Low | Age High] : transacted 6 days ago, the highest average spend and the highest limit, the lowest debt burden at 0,07, median age 64 and 70,0% retired. The ideal customer of the portfolio.
2. Cluster 1 - Young Light Users [R Low | F Low | M Med | Limit Low | DTI Low | Age Low] : no transaction for 604 days, yet still holds Rp 68,8 juta of approved limit on average and a low DTI of 0,13. Approved capacity that is simply not being used.
3. Cluster 2 - Leveraged Heavy Spenders [R High | F High | M High | Limit High | DTI High | Age High] : just as active as Cluster 0 and the largest group at 36,9%, but carries a DTI of 0,34 and the highest fraud rate at 0,25%. The biggest revenue source and the biggest exposure at once.
4. Cluster 3 - High-Burden Low Spenders [R Low | F Low | M Low | Limit Low | DTI High | Age Low] : dormant as well, with the smallest limit, the highest DTI at 0,35 and the youngest median age at 25. Inactive and already leveraged.



Cluster Centres (Visualisation & Insight)

Visualisation



What the chart shows

Green is above the portfolio average on that feature, red below. Values are the scaled cluster centres.

Activity is the first split: C0 and C2 are red on recency (recent) and green on frequency and value, while C1 and C3 are the mirror image.

Debt burden is the second: C0 sits at -1,12 on DTI against +0,54 for C2, and C1 at -0,69 against +0,65 for C3.

Insight

The model found two axes rather than one: active versus dormant, and low versus high debt burden. That produces one healthy and one leveraged segment on each side, which is exactly the split needed to decide who can be encouraged to spend more and who cannot.



Segment Value & Prioritisation

Cluster & Persona	MDR revenue	Share	Revenue / customer	Spend-to-limit	DTI	Growth score
2 · Leveraged Heavy Spenders	Rp 3,82 M	55,9%	Rp 5.347.732	5,18x	0,34	0,02
0 · High-Value Active Spenders	Rp 3,01 M	44,1%	Rp 6.292.899	3,73x	0,07	0,57
1 · Young Light Users	Rp 25.146	0,0%	Rp 99	0,00x	0,13	0,91
3 · High-Burden Low Spenders	Rp 0	0,0%	Rp 0	0,00x	0,35	0,60

Insight

All MDR revenue comes from the two active segments, and the larger of them is also the more leveraged. The growth score ranks Young Light Users first because it holds approved capacity at a DTI of 0,13, well under the portfolio median of 0,27. High-Burden Low Spenders ranks second on capacity alone and is excluded anyway: at a DTI of 0,35 it sits above that median, so activating it would convert revenue into credit risk on a 1,5% margin.

38,4%

of customers made no transaction in 6 months

Rp 34,9 M

of approved limit idle in those two segments

55,9%

of MDR revenue from one segment at DTI 0,34

Summary of Milestone 3

- Method: k-means on the three RFM columns plus credit_limit, DTI and age, run on 1.938 users after PowerTransformer scaling, which cut the worst skew from 3,06 to 0,05.
- Number of clusters: the elbow bends between k = 3 and k = 4, and the silhouette score is 0,3765 at k = 3 against 0,3001 at k = 4. k = 4 is chosen because it separates the dormant customers by debt burden, which k = 3 hides.
- The portfolio splits on two axes, not one: active versus dormant, and low versus high DTI. That gives one healthy and one leveraged segment on each side.
- All MDR revenue comes from the two active segments: Leveraged Heavy Spenders 55,9% and High-Value Active Spenders 44,1%. The second returns more per customer (Rp 6,29 juta against Rp 5,35 juta) at a far lower DTI.
- 744 customers, 38,4% of the base, made no transaction at all in six months while holding roughly Rp 34,9 miliar of approved credit limit between them.
- Only one of the two dormant segments is safe to activate: Cluster 1 sits at a DTI of 0,13 against a portfolio median of 0,27, while Cluster 3 sits at 0,35.

k = 4

chosen, silhouette 0,3001

744

dormant customers, zero transactions

Rp 261 jt

potential MDR if Cluster 1 spends 1x its limit

e Recommendations for the Performance Management Lead

Priority	Action	Why (evidence)	Expected effect
1. Reactivate the dormant low-risk segment	Win-back offer for Young Light Users (253)	0 transactions; DTI 0.13; Rp68.8M limit/user	About Rp 261 juta of extra MDR revenue if the segment spends one times its limit, with no new credit granted.
2. Protect the two active segments	Premium tier, priority servicing and proactive limit reviews so a transaction is never declined for headroom.	They produce 100% of MDR revenue; Cluster 0 alone returns Rp 6,29 juta per customer at a DTI of 0,07.	Retention of all current revenue, which is worth more than any activation campaign.
3. Grow the leveraged segment carefully	Keep promos for Leveraged Heavy Spenders volume-based rather than limit-based, and pair them with velocity checks.	Largest revenue share at 55,9%, but a DTI of 0,34 and the highest fraud rate in the portfolio at 0,25%.	More transaction volume without raising default or fraud exposure on a 1,5% margin.
4. Do not activate the high-burden dormant segment	Exclude High-Burden Low Spenders (491 users) from spend promos and offer restructuring and a limit review instead.	DTI 0,35 against a portfolio median of 0,27, the smallest limit, the youngest group and zero transactions.	Avoids turning a dormant segment into new bad debt.
5. Target on behaviour, not on age	Use recency and DTI as the targeting keys instead of age or retirement status.	The best segment is the oldest (median 64, 70,0% retired, DTI 0,07) while the riskiest is the youngest (median 25).	Campaigns land on customers who can pay rather than on a demographic proxy.

Thank You!!!

Reinaldy Zulfananda Arkaan

RevoU Full Stack Data Analytics | [W7W8W9 JUN26] | Intermediate

LinkedIn: <https://www.linkedin.com/in/reinaldy-zulfananda-arkaan-30567a179>

Google Colab: <https://colab.research.google.com/drive/1laXcBLqGQ3wUA5VCGSwyp2JRQMFFfC7Q?usp=sharing>